

Software Engineering

Dr. Md. Nasim Adnan

A Generic Process Model

- As I discussed in Chapter 1, a generic process framework for software engineering defines five framework activities—communication, planning, modeling, construction, and deployment.
- In addition, a set of umbrella activities—project tracking and control, risk management, quality assurance, configuration management, technical reviews, and others—are applied throughout the process.

Generic Process Model

A Generic Process Model is a general framework that outlines the common stages in software development, regardless of the specific methodology used. It serves as a standard guideline to ensure software is built in a structured and organized way. Following this model helps maintain quality and consistency across different projects.

Communication

The process begins with communication, where developers interact with stakeholders to understand the exact requirements of the software. This step is crucial to avoid misunderstandings and ensure the project starts with a clear vision. Effective communication at this stage builds trust and aligns expectations.

Planning

In this stage, the scope of the project is defined, deadlines are set, resources are allocated, and potential risks are identified. A strong planning phase helps maintain efficiency and reduces the chance of delays. Proper planning also enables better decision-making during the project.

Modeling

This phase focuses on designing the system's architecture and detailed components. Standardized design practices at this stage help create a system that is maintainable and scalable. A well-prepared design makes the development process smoother and more predictable.

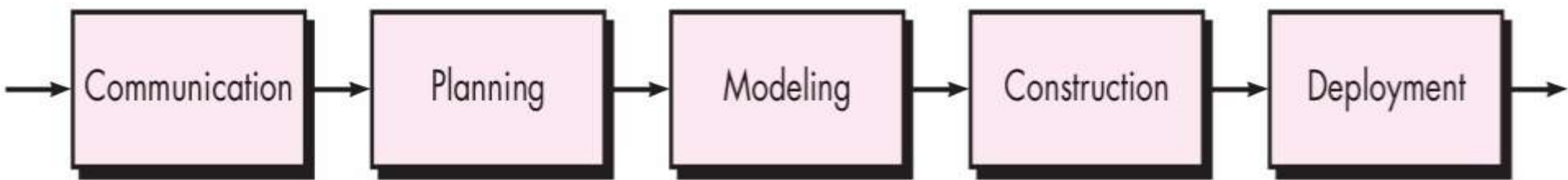
Construction

Here, the actual coding of the system takes place based on the design documents. Following standard coding and testing practices ensures quality and reduces future maintenance costs. Careful development in this stage minimizes errors and improves system stability.

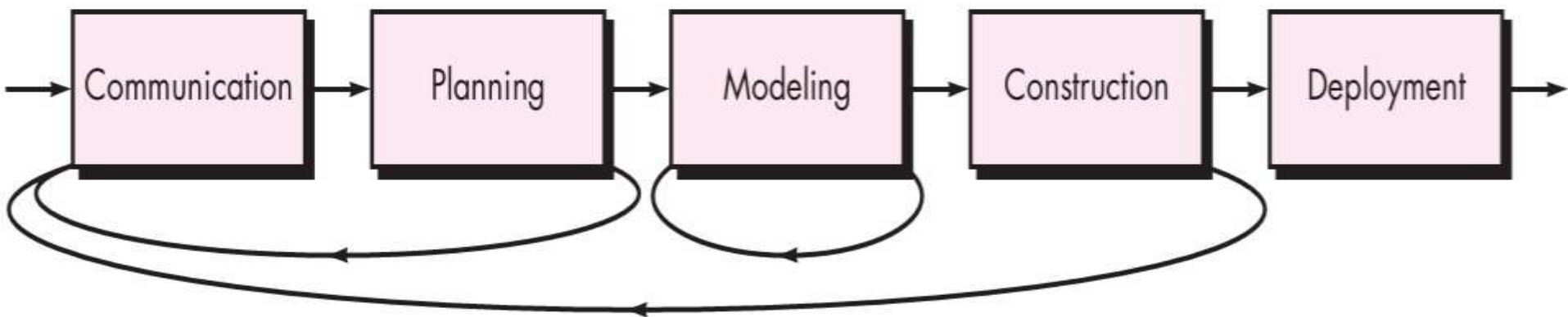
Deployment

After development and testing, the software is delivered to users. Standard deployment processes make sure the product is stable and ready for real-world use. A well-managed deployment increases user satisfaction and adoption rates.

- You should note that one important aspect of the software process has not yet been discussed.
- This aspect—called **process flow**—describes how the framework activities and the actions and tasks that occur within each framework activity are organized with respect to sequence and time and is illustrated in the next figure.

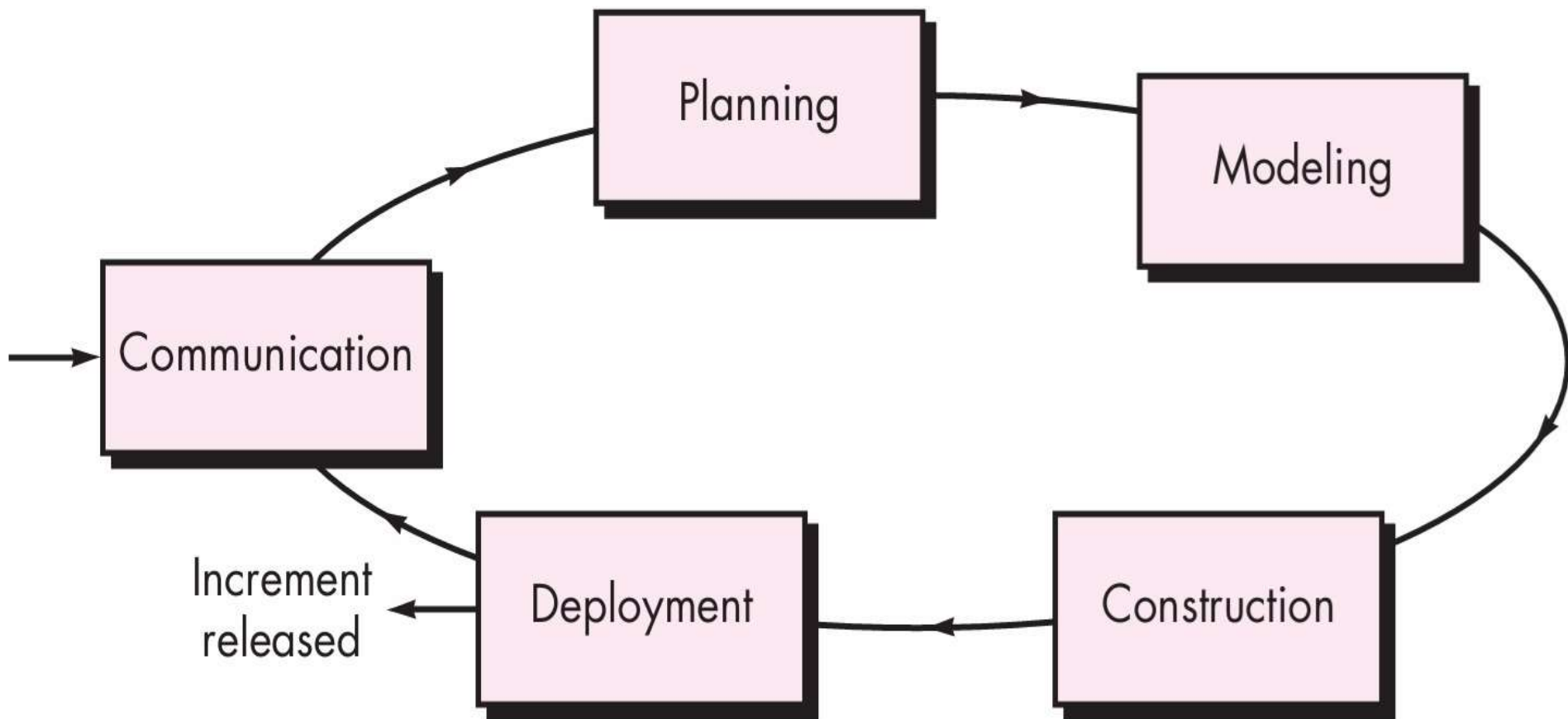


(a) Linear process flow

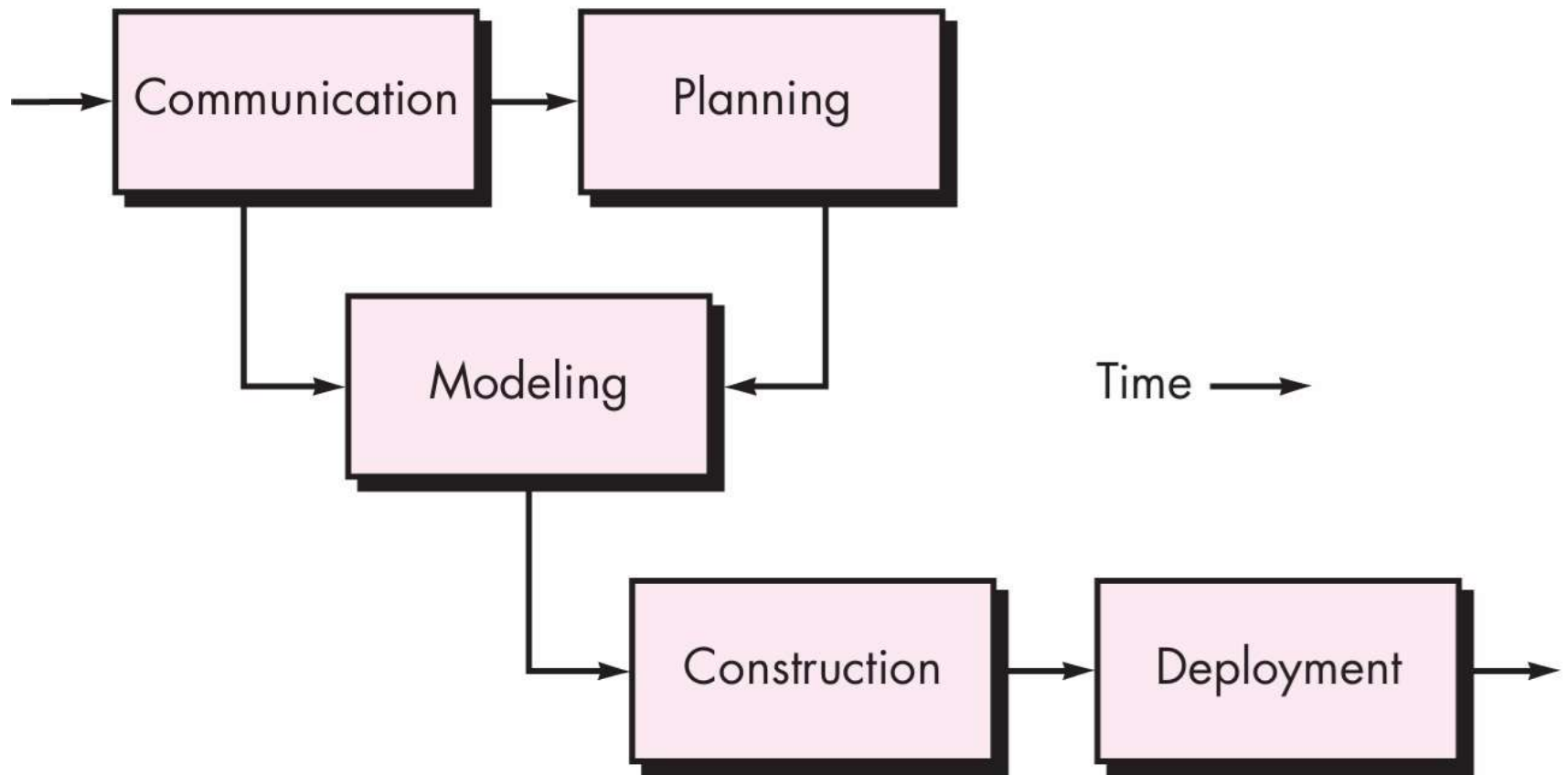


(b) Iterative process flow

- A linear process flow executes each of the five framework activities in sequence, beginning with communication and ending with deployment (Figure a).
- An iterative process flow repeats one or more of the activities before proceeding to the next (Figure b).
- An evolutionary process flow executes the activities in a “circular” manner. Each circuit through the five activities leads to a more complete version of the software (Figure c).
- A parallel process flow (Figure d) executes one or more activities in parallel with other activities (e.g., modeling for one aspect of the software might be executed in parallel with construction of another aspect of the software).



(c) Evolutionary process flow



(d) Parallel process flow

Perspective Process Models

- Prescriptive models were made to make software development more organized and less confusing.
- In the past, these models gave software teams a clear structure and a step- by- step guide (a roadmap) to follow during projects.
- But even with these models, software development can still feel messy and unpredictable — it often stays on “the edge of chaos.”

The Waterfall Model

The Waterfall Model works best when the project requirements are clear and well understood.

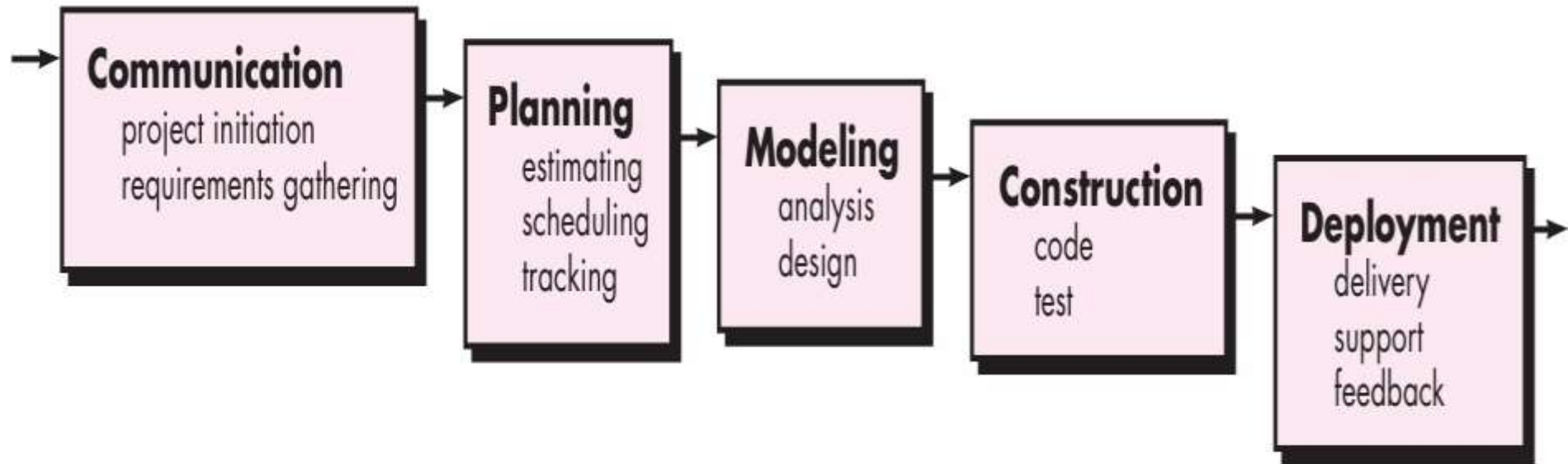
In such cases, the work moves in a straight, step-by-step order – from talking with users (communication) to giving them the final product (deployment).

This model is useful when:

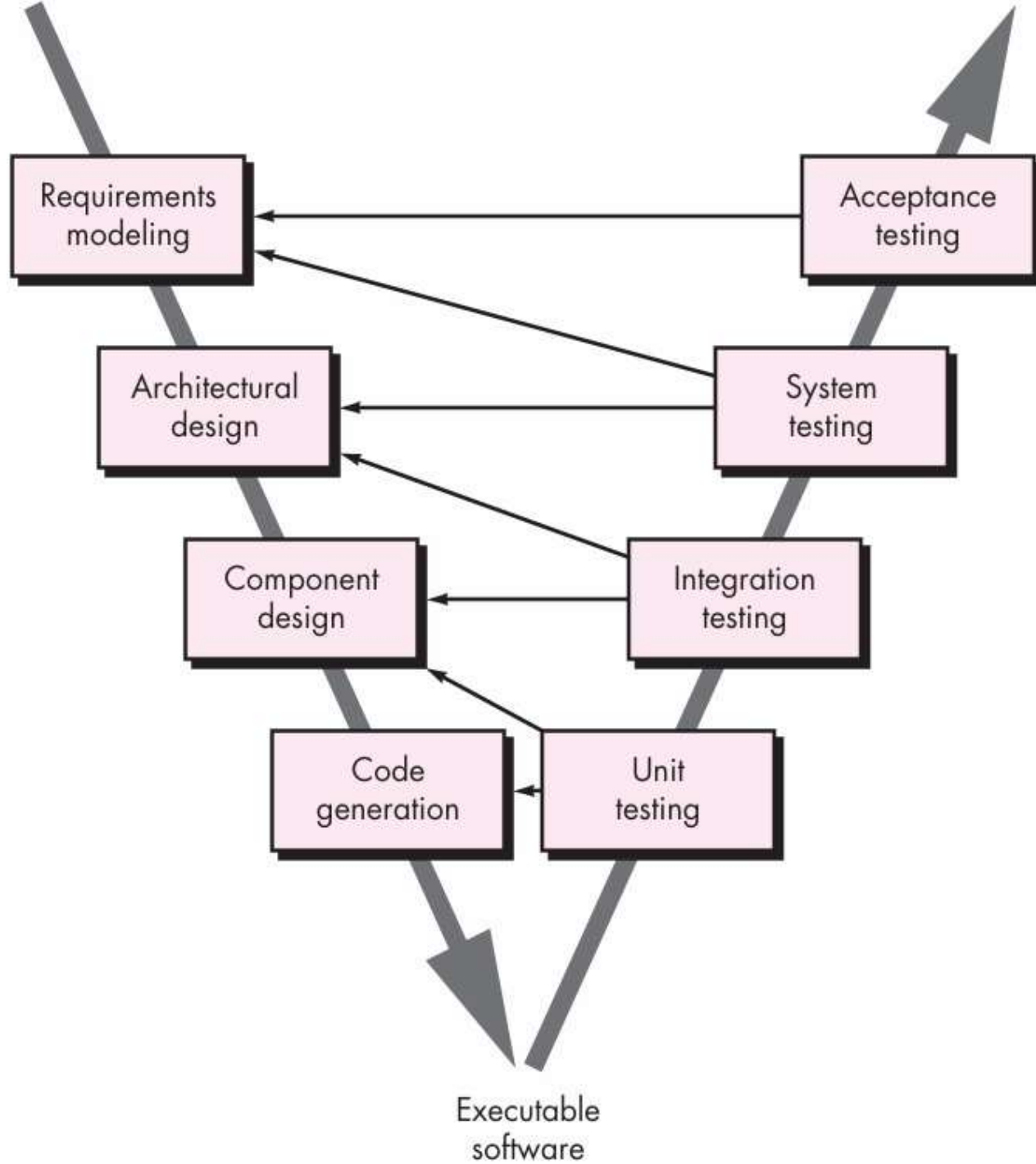
- You are making specific changes to an existing system.

- Example: Updating accounting software because of new government rules.

- In a few new projects, if the needs are clearly defined and unlikely to change.



- A variation in the representation of the waterfall model is called the V-model.
- The V-model (shown in the next figure) shows how quality assurance steps are connected to steps like planning (communication), designing (modeling), and early building (construction) of the software.



As a software team goes down the left side of the V-model, they take the basic problem and turn it into more detailed technical designs.

After writing the code, the team moves up the right side of the V, testing the system step by step to make sure each part works correctly based on the earlier designs.

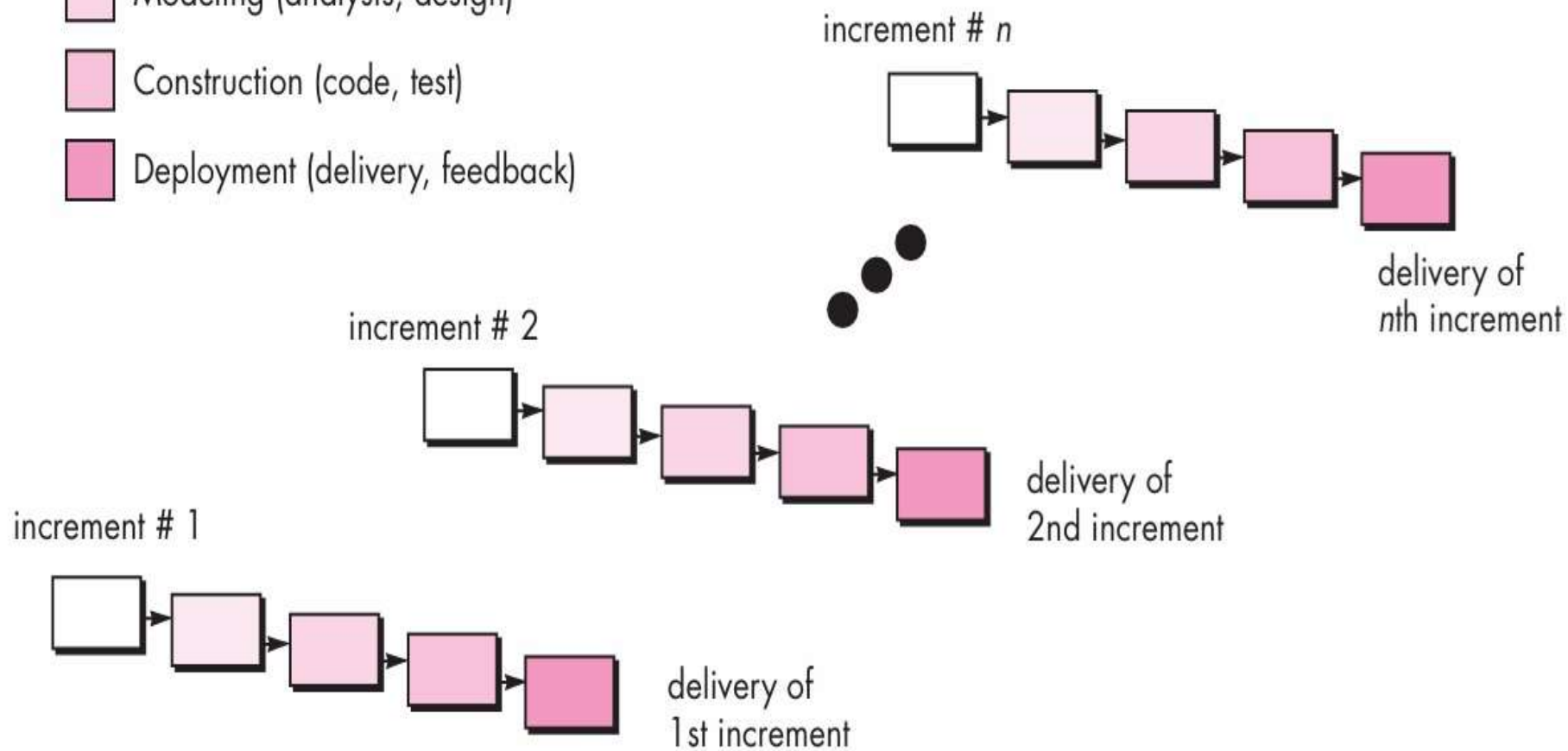
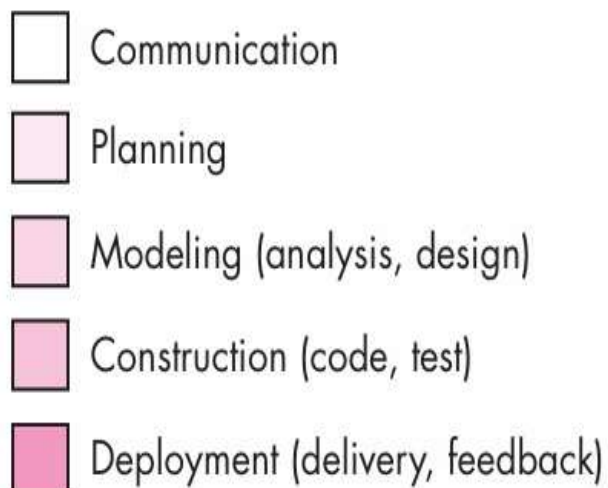
In fact, the V-model is very similar to the classic Waterfall model. It just gives a clearer picture of how testing (verification and validation) is connected to each earlier step in development.

Incremental Process Models

The Incremental Process Model is a way of building software step by step. Instead of making the whole system at once, the software is developed in small parts called increments. The first version includes only the most important features (called the core product). Later, more features are added in future versions.

This approach helps users get a basic working version quickly, and new features are added over time based on feedback.

Software Functionality and Features



Project Calendar Time

1. What is Delivered First in the Incremental Model

When the Incremental Model is used, the first thing built is usually the core product. This core product covers only the basic features needed to start using the software. Some extra features, whether already known or not yet discovered, are not included at this stage.

The customer uses the core product or reviews it carefully. Based on how the product performs and what the customer wants, a plan is made for the next version (increment). This plan includes improving the core product and adding new features.

This process is repeated again and again until the full product is finished.

2. Why the Incremental Model is Useful

With each new increment, the customer receives a working version of the software. Early versions may be simpler than the final product, but they are still useful. They help the customer test the software early and give feedback.

This model is especially helpful when there are not enough team members to build the whole software at once, or when there is a strict deadline. A small team can create the first version. If the core product is successful, more people can join later to build the next versions.

The incremental model allows faster delivery, better testing, and flexibility to adjust based on customer needs.

Evolutionary Process Models

Software, like any complex system, changes and improves over time as it is developed. As the project progresses, the business needs and product requirements often change. This makes it hard to follow a straight path to the final product.

** Business deadlines might force the team to release a limited version of the software before it's fully completed.

** Core features may be clear from the start, but extra features or details may still need to be figured out.

Because of these reasons, Evolutionary Process Models are used. These models work by improving and building upon previous versions of the software.

How Evolutionary Models Work:

**They are iterative, meaning the software is developed in cycles. Each cycle creates a more complete version of the software.

**The first version may only include basic features, and with each new version, more features and improvements are added.

This approach helps deal with changing requirements and allows the software to improve over time until it reaches the final version.

Prototyping

Sometimes, customers know what they want in general, but they don't give clear details about all the features or functions of the software.

In such cases, the Prototyping Model is used. It helps both developers and customers understand the system better by building a simple version of the software early.

How It Works:

1. The process begins with communication.

You meet with the customer and other people involved to:

- * Set the main goals of the software,
- * Note any known requirements,
- * Identify parts that still need to be clearly defined.

2. A quick design is created.

This design is not complete, but just enough to build a basic version of the software.

3. Using this design, a prototype is built.

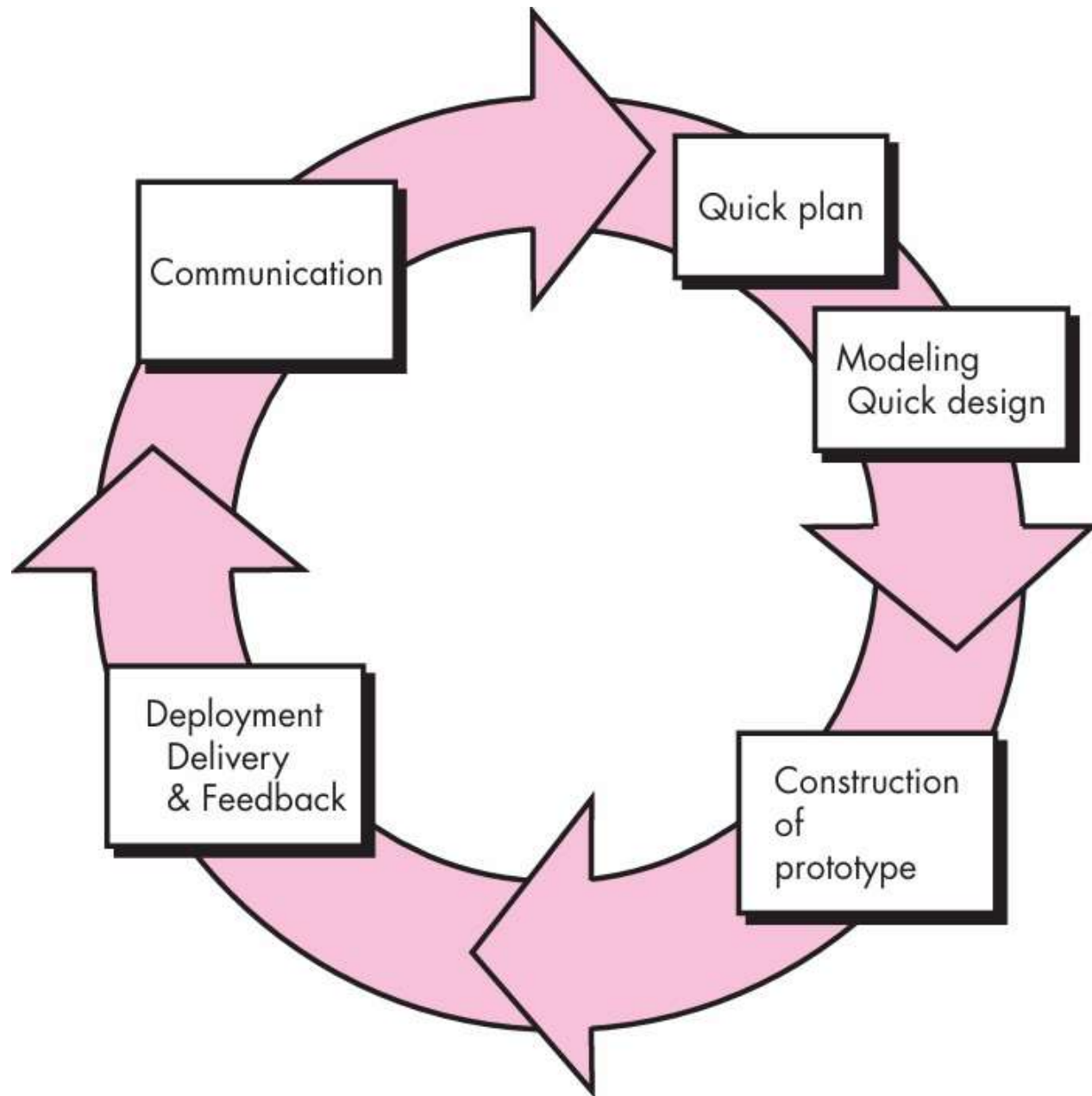
This is a simple, working model of the software — not the final version.

4. The prototype is then given to users to try out.

5. Users provide feedback, saying what they like, what they don't, and what they still need.

6. Based on this feedback, the prototype is improved.

7. This process continues, with multiple updates, until everyone understands the requirements well and the developers know exactly what to build.



The Spiral Model

The Spiral Model is a software development method that builds the project in steps or cycles.

It was created by Barry Boehm.

This model combines:

- The trial-and-error approach of prototyping, and
- The structured step-by-step approach of the Waterfall model.

How it works:

1. The project is developed in many loops (spirals).

Each loop is like a phase of the software.

One loop = one version of the software (like a draft).

2. Each loop has 5 main steps:

Planning – Decide what to do and gather requirements.

Risk Analysis – Find possible problems or risks and think of solutions.

Engineering - Build a part of the software (prototype or real feature).

Evaluation – Test and get feedback from users or clients.

Deployment – Deliver the current version and use feedback to improve in the next loop.

3.After each loop:

A better version of the software is created.

It could be a prototype in early loops, or

A more complete system in later loops.

Planning

estimation
scheduling
risk analysis

Modeling

analysis
design

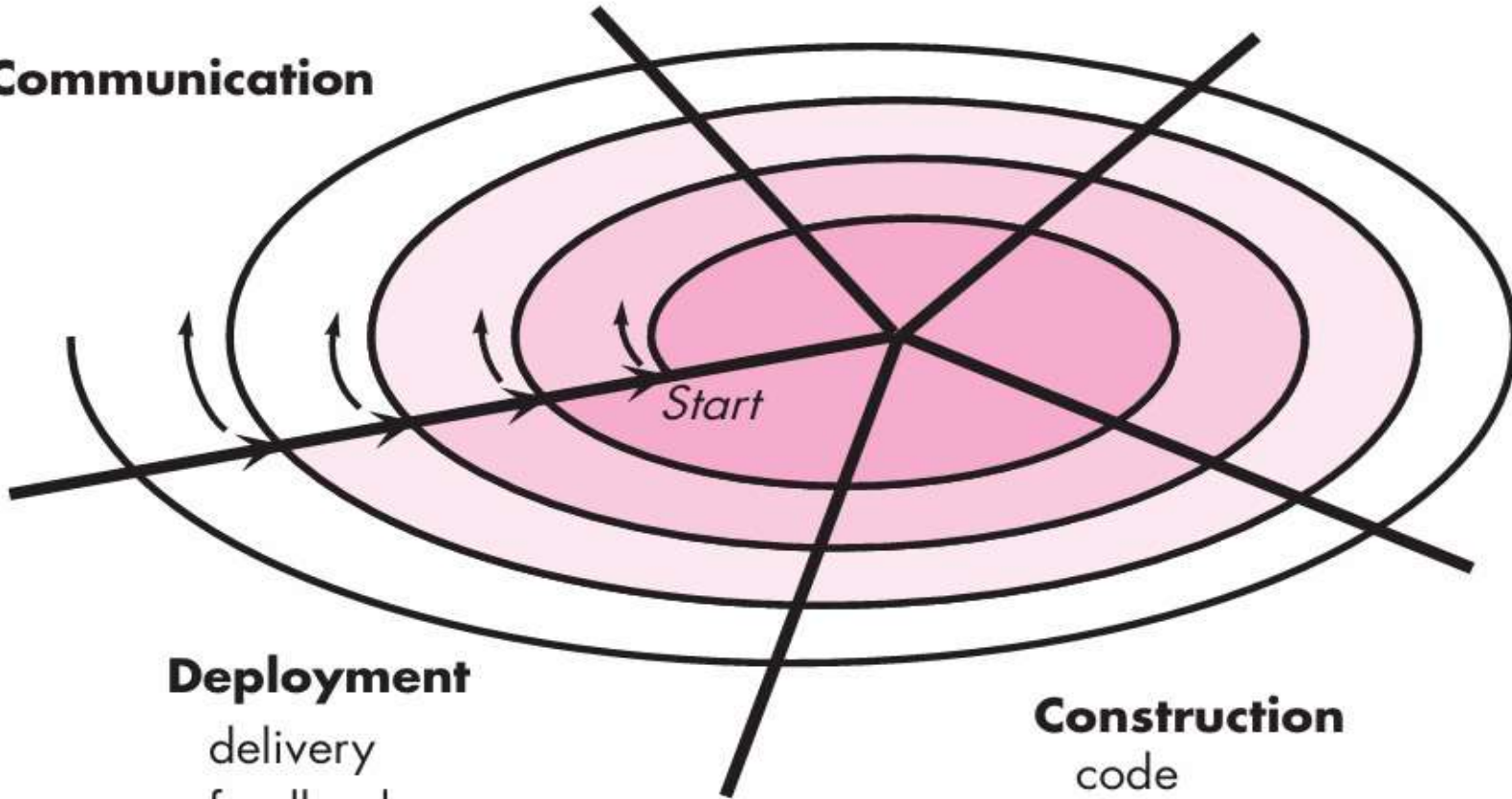
Construction

code
test

Deployment

delivery
feedback

Communication



Concurrent Models

- The concurrent development model, sometimes called concurrent engineering, allows a software team to represent iterative and concurrent elements of any of the process models.

Specialized Process Models:

Component-Based Development

- Commercial off-the-shelf (COTS) software components, developed by vendors who offer them as products, provide targeted functionality with well-defined interfaces that enable the component to be integrated into the software that is to be built.
- The component-based development model incorporates many of the characteristics of the spiral model. It is evolutionary in nature, demanding an iterative approach to the creation of software.

The Formal Methods Model

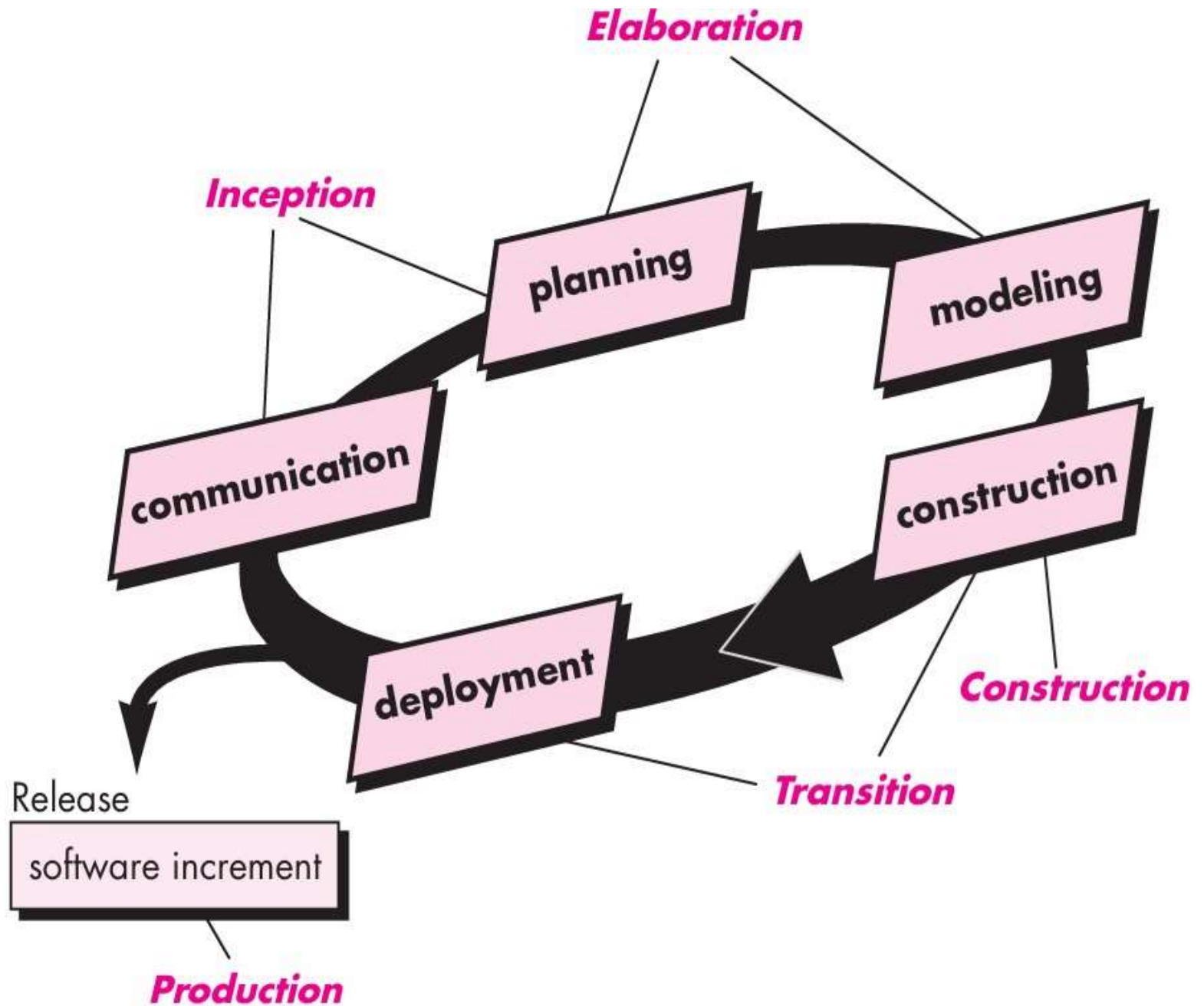
- The formal methods model encompasses a set of activities that leads to formal mathematical specification of computer software.
- Formal methods enable you to specify, develop, and verify a computer-based system by applying a rigorous, mathematical notation.

Aspect-Oriented Software Development

- When concerns cut across multiple system functions, features, and information, they are often referred to as crosscutting concerns.
- Aspectual requirements define those crosscutting concerns that have an impact across the software architecture (such as logging, security, error handling etc.).
- Aspect-oriented software development (AOSD), often referred to as aspect-oriented programming (AOP), is a relatively new software engineering paradigm that provides a process and methodological approach for defining, specifying, designing, and constructing aspects.

The Unified Process Model

- This model uses UML—a Unified Modeling Language that contains a robust notation for the modeling and development of object-oriented systems.
- By 1997, UML became a de facto industry standard for object-oriented software development.



Personal and Team Process Models

- The best software process is one that is close to the people who will be doing the work.
- If a software process model has been developed at a corporate or organizational level, it can be effective only if it is amenable to significant adaptation to meet the needs of the project team that is actually doing software engineering work.
- In an ideal setting, you would create a process that best fits your needs, and at the same time, meets the broader needs of the team and the organization.

- Alternatively, the team itself can create its own process, and at the same time meet the narrower needs of individuals and the broader needs of the organization.
- It is possible to create a “personal software process” and/or a “team software process.”
- Both require hard work, training, and coordination, but both are achievable.

